

Django All Topics

1) Settings

import os

```
INSTALLED_APPS = [  
    'myapp',  
]
```

templates

```
'DIRS': [os.path.join(BASE_DIR, 'templates')],
```

static files

```
STATICFILES_DIRS=[os.path.join(BASE_DIR, 'static')]
```

#media

```
MEDIA_URL = '/media/'  
MEDIA_ROOT = os.path.join(BASE_DIR, 'media')
```

Connecting Mysql Database

```
DATABASES = {  
    'default': {  
        'ENGINE': 'django.db.backends.mysql', # MySQL backend  
        'NAME': 'your_db_name',             # Database name  
        'USER': 'root',                      # MySQL username  
        'PASSWORD': 'your_password',        # MySQL password  
        'HOST': 'localhost',                 # Or IP address like '127.0.0.1'  
        'PORT': '3306',                     # Default MySQL port  
        'OPTIONS': {  
            'init_command': "SET sql_mode='STRICT_TRANS_TABLES'"  
        },  
    },  
}
```

User Model Settings

```
AUTH_USER_MODEL = 'myapp.MyUser'
```

Note :Important: Set AUTH_USER_MODEL before the first migration, otherwise you'll face migration issues.

Set login redirect URL

```
LOGIN_REDIRECT_URL = '/'  
LOGOUT_REDIRECT_URL = '/'
```

2) Django Live Reload

1. Install the package

`pip install django-browser-reload`

2. Add it to INSTALLED_APPS in settings.py

```
INSTALLED_APPS = [  
    "django_browser_reload",  
]
```

3. Add middleware in settings.py

Place it after

`django.middleware.common.CommonMiddleware:`

```
MIDDLEWARE = [  
    "django_browser_reload.middleware.BrowserReloadMiddleware",  
]
```

4. Add URL config in urls.py

from `django.urls` import `path`, `include`

```
urlpatterns = [  
    path("__reload__/", include("django_browser_reload.urls")),  
]
```

5. Use it in your template

Usually placed before the closing `</body>`:

```
{% django_browser_reload_script %}  
</body>
```

3) Django Debug Tool Bar

1. Install the package

```
pip install django-debug-toolbar
```

2. Add to INSTALLED_APPS

```
INSTALLED_APPS = [  
    # your apps  
    'debug_toolbar',  
]
```

3. Add middleware

```
MIDDLEWARE = [  
    'debug_toolbar.middleware.DebugToolbarMiddleware',  
    # other middleware...  
]
```

4. Configure internal Ips

```
settings.py  
INTERNAL_IPS = [  
    "127.0.0.1",  
]
```

5. Configure internal Ips

```
from django.urls import include, path  
urlpatterns = [  
    path('__debug__/', include('debug_toolbar.urls')),  
]
```

4) Admin.py

```
from django.contrib import admin
from django.apps import apps
from django.contrib.admin.sites import AlreadyRegistered

# Automatically register all models from the current app
app = apps.get_app_config('your_app_name')

for model in app.get_models():
    try:
        admin.site.register(model)
    except AlreadyRegistered:
        pass # If the model is already registered, skip it
```

5) Urls.py

```
from django.contrib import admin
from django.urls import path
```

```
from myapp import views
```

```
from django.conf import settings
from django.conf.urls.static import static
```

```
urlpatterns = [
    path('admin/', admin.site.urls),
    path('show/', views.show, name='show'),
    path('save/', views.save, name='save'),
    path('update/<int:id>', views.update),
    path('delete/<int:iden>', views.delete, name='delete'),
    path('sample/', views.sample, name='sample2'),
]
```

```
if settings.DEBUG:
```

```
    urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

Forms

6) Import Statements

```
from django import forms
from django.contrib.auth.forms import AuthenticationForm
from django.core.exceptions import ValidationError
from .models import User,transaction_model
```

7) Example-1

```
from django import forms
from django.contrib.auth.forms import AuthenticationForm
from django.core.exceptions import ValidationError
from .models import User,transaction_model

class user_model_form(forms.ModelForm):
    dob=forms.DateField(widget=forms.DateInput(attrs={'type':'date'}))
    gender_choice=[('Male','Male'),('Female','Female')]
    gender=forms.ChoiceField(widget=forms.RadioSelect,choices=gender_choice)
    retypepassword=forms.CharField(max_length=20)
    class Meta:
        model = User
        fields = ('username',
'customer_name','password','retypepassword','dob','gender','mobilen','email','address','photo','accno')

        help_texts = {
            'username': None,
        }

    def clean_customer_name(self):
        customer_name = self.cleaned_data.get('customer_name')
        if not customer_name.isalpha(): # Custom validation
            raise forms.ValidationError("customer_name should only contain letters.")
        return customer_name

    def clean(self):
        cleaned_data = super().clean()
        password = cleaned_data.get('password')
        retypepassword = cleaned_data.get('retypepassword')

        if password!=retypepassword:
            self.add_error('password','Password and Retypepassword not Match')
            #raise ValidationError("Password and RetypePassword are not Macth")
        return cleaned_data

    def clean_mobilen(self):
        mobilen = self.cleaned_data.get('mobilen')
        if not mobilen.isdigit() or len(mobilen) != 10:
            raise forms.ValidationError("Mobile number must be exactly 10 digits.")
        return mobilen
```

```
def save(self, commit=True):
    user = super().save(commit=False)
    user.set_password(self.cleaned_data["password"])
    if commit:
        user.save()
    return user
```

```
#-----
```

```
class CustomAuthenticationForm(AuthenticationForm):
```

```
    username = forms.CharField(
        widget=forms.TextInput(attrs={'class': 'form-control', 'placeholder': 'Username'}),
        label="Username"
    )
    password = forms.CharField(
        widget=forms.PasswordInput(attrs={'class': 'form-control', 'placeholder': 'Password'}),
        label="Password"
    )
```

```
#-----
```

```
class amount_modelform(forms.ModelForm):
```

```
    class Meta:
        model = transaction_model
        fields = ('amount',)
```

8) Example-2

```
class students_form(forms.Form):
    Integerfield=forms.IntegerField(required=False)
    Decimalfield=forms.DecimalField(required=False)
    Charfield=forms.CharField(max_length=20,label='Enter Name',required=False,initial='Enter Name')
    Password=forms.CharField(max_length=20,widget=forms.PasswordInput,required=False)
    Textarea=forms.CharField(widget=forms.Textarea(attrs={'rows':5,'cols':30}),required=False)
    Date_1=forms.DateField(required=False)
    Date_2=forms.DateField(widget=NumberInput(attrs={'type':'date'}),required=False)
    year=['1990','2000','2010','2020']
    Date_3=forms.DateField(widget=forms.SelectDateWidget(years=year),required=False)
    Date_4=forms.DateField(initial=datetime.date.today,required=False)
    DateTimefield=forms.DateTimeField(initial=datetime.date.today,required=False)

    Emailfield=forms.EmailField(required=False)

    data_base_1=((---Select---','---Select---'),('Mysql','Mysql'),('Sqlite','Sqlite'),('Oracle','Oracle'))
    Dropdownlist=forms.ChoiceField(choices=data_base_1,required=False)
    data_base_2=((('Mysql','Mysql'),('Sqlite','Sqlite'),('Oracle','Oracle'))
    Radiobutton=forms.ChoiceField(widget=forms.RadioSelect,choices=data_base_2,required=False)

    Checkbox_1=forms.MultipleChoiceField(widget=forms.CheckboxSelectMultiple,choices=data_base_2,
required=False)
    Listbox=forms.MultipleChoiceField(choices=data_base_2,required=False)

    Modeldropdownlist=forms.ModelChoiceField(queryset=choice_model.objects.all(),initial=0,required=False)

    Modelradiobutton=forms.ModelChoiceField(widget=forms.RadioSelect,queryset=choice_model.objects
.all(),initial=0,required=False)

    Modelcheckbox=forms.ModelMultipleChoiceField(widget=forms.CheckboxSelectMultiple,queryset=ch
oice_model.objects.all(),initial=0,required=False)

    Modellistbox=forms.ModelMultipleChoiceField(queryset=choice_model.objects.all(),initial=0,required
=False)

    Booleanfield=forms.BooleanField(required=False)
    Imagefield=forms.ImageField(required=False)
    Filefield=forms.FileField(required=False)
```


Views

9) Import Statements

```
from django.shortcuts import HttpResponseRedirect, render, redirect
```

```
from .forms import *  
from .models import *  
from .forms import StudentForm  
from .models import StudentsInfoModel
```

```
from django.contrib.auth.models import User  
from django.contrib.auth import authenticate, logout, login  
from django.contrib.auth.decorators import login_required
```

```
from django.contrib import messages  
from django.core.exceptions import ValidationError
```

```
from django.views.generic.list import ListView  
from django.views.generic.edit import CreateView, UpdateView, DeleteView
```

```
from django.core.mail import send_mail
```

```
from django.db.models import Q  
from django.db.models import IntegerField  
from django.db.models.functions import Cast  
from datetime import date, datetime  
from django.db.models import Sum, Avg, Max, Min, Count
```

10) Signup (Using HTML Form)

```
def Signup(request):  
    if request.method=='POST':  
        username=request.POST['username']  
        password1=request.POST['password1']  
        password2=request.POST['password2']  
        if password1==password2:  
            try:  
                User.objects.create_user(username=username,password=password1)  
                messages.success(request,'Successfully Register')  
                return redirect('/Login')  
            except Exception as e:  
                messages.error(request,'Username Already Exists')  
        return render(request,'user/Signup.html')
```

11) Login (Using HTML Form)

```
def Login(request):  
    if request.method=='POST':  
        username=request.POST['username']  
        password=request.POST['password']  
        user=authenticate(username=username,password=password)  
        if user:  
            login(request,user)  
            return redirect('/Home')
```

```
else:
    messages.error(request,'Invalid UserName and Password')
return render(request,'user/Login.html')
```

12) Logout

```
def Logout(request):
    logout(request)
    return redirect('/Login')
```

13) Password Change(Using HTML Form)

```
def ChangePassword(request):
    if request.method=='POST':
        oldpassword=request.POST['oldpassowrd']
        newpassword=request.POST['newpassword']
        confirm_new_password=request.POST['confirm_new_password']
        if newpassword==confirm_new_password:
            current_user=request.user
            user=authenticate(username=current_user,password=oldpassword)
            if user:
                user.set_password(newpassword)
                user.save()
                messages.success(request,'Password Succssfully Changed')
            return redirect('/Login')
    return render(request,'user/ChangePassword.html')
```

14) Context

```
def Department_Wise_Strength(request):
    cse_male_noofstudents=StudentsInfoModel.objects.filter(department='CSE',gender='Male').count()

    cse_female_noofstudents=StudentsInfoModel.objects.filter(department='CSE',gender='Female').count()
    cse_noofstudents=StudentsInfoModel.objects.filter(department='CSE').count()

    aids_male_noofstudents=StudentsInfoModel.objects.filter(department='AIDS',gender='Male').count()

    aids_female_noofstudents=StudentsInfoModel.objects.filter(department='AIDS',gender='Female').count()
    aids_noofstudents=StudentsInfoModel.objects.filter(department='AIDS').count()

    context={
        'cse_male_noofstudents':cse_male_noofstudents,
        'cse_female_noofstudents':cse_female_noofstudents,
        'cse_noofstudents':cse_noofstudents,
        'aids_male_noofstudents':aids_male_noofstudents,
        'aids_female_noofstudents':aids_female_noofstudents,
        'aids_noofstudents':aids_noofstudents,
    }

    return render(request,'Department_Wise_Strength.html',{'context':context})
```

15) Show Database Data

```
def show(request):
    data=students.objects.all()
    return render(request,'show.html',{'data':data})
```

16) Save(Using HTML Form)

```
def save(request):
    if request.method=='POST':
        regno=request.POST['regno']
        name=request.POST['name']
        dob=request.POST['dob']
        gender=request.POST['gender']
        skill=request.POST.getlist('skill[]')
        skill_data=""
        for s1 in skill:
            skill_data = skill_data +s1 +", "

        department=request.POST['department']
        email=request.POST['email']

        obj=students()
        obj.regno=regno
        obj.name=name
        obj.dob=dob
        obj.gender=gender
        obj.skill=skill_data
        obj.department=department
        obj.email=email
        obj.save()
        return redirect('/show')
    return render(request, 'index.html')
```

17) Update(Using HTML Form)

```
def update(request,id):
    obj=students.objects.get(id=id)
    skill_list=obj.skill.split(',')
    if request.method=='POST':
        obj.regno=request.POST['regno']
        obj.name=request.POST['name']
        obj.dob=request.POST['dob']
        obj.gender=request.POST['gender']
        skill=request.POST.getlist('skill[]')
        skill_data=""
        for s1 in skill:
            skill_data = skill_data +s1 +", "
        obj.skill=skill_data
        obj.department=request.POST['department']
        obj.email=request.POST['email']
        obj.save()
        return redirect('/show')
    return render(request, 'update.html',{'data':obj,'skill_list':skill_list})
```

18) Delete

```
def delete(request,id):  
    data=students.objects.get(id=id)  
    data.delete()  
    return redirect('/show')
```

19) Save(Using Model Form)

```
def save(request):  
    if request.method=='POST':  
        form=student_modelform(request.POST,request.FILES)  
        if form.is_valid():  
            form.save()  
            return redirect('/show')  
    else:  
        #send already filled form data and error message  
        return render(request,'save.html',{'form':form})  
    form=student_modelform()  
    return render(request,'save.html',{'form':form})
```

20) Update(Using Model Form)

```
def update(request,id):  
    data=students.objects.get(id=id)  
    form=student_modelform(instance=data)  
    if request.method=='POST':  
        form=student_modelform(request.POST,request.FILES,instance=data)  
        if form.is_valid():  
            form.save()  
            return redirect('/show')  
    else:  
        #send already filled form data and error message  
        return render(request,'save.html',{'form':form})  
    return render(request,'save.html',{'form':form})
```

21) Django ORM

1. Model

```
from django.db import models
```

```
class students_model(models.Model):  
    name=models.CharField(max_length=30)  
    gender=models.CharField(max_length=10)  
    age=models.IntegerField()  
    department=models.CharField(max_length=10)  
    year=models.IntegerField()  
    place=models.CharField(max_length=10)  
    paper1=models.CharField(max_length=10)  
    paper2=models.CharField(max_length=10)
```

```
def __str__(self):  
    return self.name
```

```
class studentinfo(models.Model):  
    regno=models.IntegerField()  
    name=models.CharField(max_length=30,blank=True,null=True)  
    gender=models.CharField(max_length=10,blank=True,null=True)  
    age=models.IntegerField()
```

```
class exam_marks(models.Model):  
    regno=models.ForeignKey(studentinfo,on_delete=models.PROTECT,related_name='exam')  
    exam_choice=(('sem1','sem1'),('sem2','sem2'))  
    exam=models.CharField(max_length=10,choices=exam_choice)  
    paper1=models.CharField(max_length=10)  
    paper2=models.CharField(max_length=10)  
    result=models.CharField(max_length=10)
```

```
class Meta:  
    unique_together=('regno', 'exam')
```

```
def __str__(self):  
    return str(self.regno.regno)+'-'+self.exam
```

2. Views

```
from django.shortcuts import render,redirect
from.models import*
from django.db.models import Q
from django.db.models import Sum,Max,Min,Count,Avg
from django.db.models import IntegerField
from django.db.models.functions import Cast
from.forms import student_form
from django.contrib import messages
from django.core.mail import send_mail

def index(request):
    data=students_model.objects.all()
    data=students_model.objects.get(id=1)
    data=students_model.objects.filter(age=25).values_list()
    data=students_model.objects.filter(age__gt=25).values_list()
    data=students_model.objects.filter(age__gte=25).values_list()
    data=students_model.objects.filter(age__lt=25).values_list()
    data=students_model.objects.filter(age__lte=25).values_list()
    data=students_model.objects.exclude(age=25).values_list()
    data=students_model.objects.filter( Q() & Q())
    data=students_model.objects.filter(department='ECE',year=1).values_list()
    data=students_model.objects.filter( Q(department='ECE') & Q(year=1)).values_list()
    data=students_model.objects.filter( Q(department='ECE') | Q(year=1)).values_list()
    data=students_model.objects.order_by('name','age').values_list()
    data=students_model.objects.order_by('name').values_list()
    data=students_model.objects.order_by('-name').values_list()
    data=students_model.objects.filter(name__exact='Ragu').values_list()
    data=students_model.objects.filter(name__iexact='ragu').values_list()
    data=students_model.objects.filter(name__contains='k').values_list()
    data=students_model.objects.filter(name__startswith='ka').values_list()
    data=students_model.objects.filter(name__endswith='ra').values_list()
    data=students_model.objects.filter(age__range=[21,26]).values_list()
    data=students_model.objects.values_list('place').distinct()
    data=students_model.objects.aggregate(Max('age'))
    data=students_model.objects.aggregate(Min('age'))
    data=students_model.objects.aggregate(Sum('age'))
    data=students_model.objects.aggregate(Avg('age'))
    data=students_model.objects.aggregate(Count('age'))

    data=students_model.objects.annotate(p1=Cast('paper1',output_field=IntegerField())).exclude(paper1='AAA').aggregate(max=Max("p1"))

    data=students_model.objects.annotate(p1=Cast('paper1',output_field=IntegerField())).exclude(paper1='AAA').aggregate(minMUM=Min("p1"))

    data=students_model.objects.annotate(p1=Cast('paper1',output_field=IntegerField())).exclude(paper1='AAA').aggregate(avg=Avg("p1"))
```

```

data=students_model.objects.annotate(p1=Cast('paper1',output_field=IntegerField())).exclude(paper1='AAA').aggregate(count=Count("p1"))

data=students_model.objects.annotate(p1=Cast('paper1',output_field=IntegerField())).exclude(paper1='AAA').aggregate(Sum=Sum("p1"))
data=students_model.objects.all().first()
data=students_model.objects.all().last()
data=students_model.objects.filter(name="Ragukanan").exists()
data=students_model.objects.filter(name="")
data=students_model.objects.filter(name__isnull=True).values_list()
data=students_model.objects.filter(name__isnull=False).values_list()

# students=studentinfo.objects.all()
# for student in students:
#     print(student.name)

exams=exam_marks.objects.all()
for exam in exams:
    print(exam.regno,exam.paper1,exam.paper2,exam.regno.age,exam.regno.gender)

students=studentinfo.objects.all()
for student in students:
    markss=student.exam.all()
    print(student.regno,student.name,[mk.paper1 for mk in markss])
form=student_form()

# if request.method=='POST':
#     form=student_form(request.POST)
#     if form.is_valid():
#         form.save()
#         messages.success(request,'Student information save Successfully')
#     else:
#         messages.error(request,'Invalid data')
#     return redirect('/index')

return render(request,'index.html',{'form':form})

```

22) Send Email

```
send_mail(  
    subject='Subject here',  
    message='test mail.',  
    from_email='@gmail.com',  
    recipient_list=['@gmail.com'],  
    fail_silently=False, )
```

23) A

1) Sweet alert cdn links

```
<script src="https://cdnjs.cloudflare.com/ajax/libs/sweetalert2/11.14.5/sweetalert2.all.min.js"></script>
<link rel="stylesheet"
href="https://cdnjs.cloudflare.com/ajax/libs/sweetalert2/11.14.5/sweetalert2.css"/>
```

2) Image Preview before submit form (using javascript)

```
{% if form.instance.photo %}
    
{% endif %}
<script>
    function photo_preview(event)
    {
        let output=document.getElementById('output')
        output.src=URL.createObjectURL(event.target.files[0])
    }
</script>
```

3) Show "success", "error", "warning", "info" Messages using sweet alert library

```
<script src="https://cdn.jsdelivr.net/npm/sweetalert2@11"></script>
<script>
    document.addEventListener("DOMContentLoaded", function () {
        {% if messages %}
            {% for message in messages %}
                Swal.fire({
                    title: "{{ message.tags|title }}",
                    text: "{{ message }}",
                    icon: "{{ message.tags }}" , // Use "success", "error", "warning", "info"
                    confirmButtonText: "OK"
                });
            {% endfor %}
        {% endif %}
    });
</script>
```

4) Delete (using submit button)

```
<form id="delete-form" method="post" action="{% url 'delete' sdata.id %}">
    {% csrf_token %}
    <button type="button" class="btn btn-danger" onclick="confirmDelete()">Delete</button>
</form>
```

```
<script src="https://cdnjs.cloudflare.com/ajax/libs/sweetalert2/11.14.5/sweetalert2.all.min.js"></script>
<link rel="stylesheet"
href="https://cdnjs.cloudflare.com/ajax/libs/sweetalert2/11.14.5/sweetalert2.css"/>
```

```
<script>
function confirmDelete() {
    Swal.fire({
        title: 'Are you sure?',
        text: "You won't be able to revert this!",
        icon: 'warning',
        showCancelButton: true,
        confirmButtonColor: '#3085d6',
        cancelButtonColor: '#d33',
        confirmButtonText: 'Yes, delete it!'
    }).then((result) => {
        if (result.isConfirmed) {
            // Submit the form programmatically
            document.getElementById('delete-form').submit();
        }
    });
}
</script>
```

5) Delete (using <a> tag)

```
<a href="#" class="btn btn-danger delete-button" data-url="{% url 'delete' sdata.id %}">
    Delete
</a>
```

```
<script src="https://cdnjs.cloudflare.com/ajax/libs/sweetalert2/11.14.5/sweetalert2.all.min.js"></script>
<link rel="stylesheet"
href="https://cdnjs.cloudflare.com/ajax/libs/sweetalert2/11.14.5/sweetalert2.css"/>
```

```
<script>
    document.addEventListener('DOMContentLoaded', function () {
        // Select all delete buttons
        document.querySelectorAll('.delete-button').forEach(button => {
            button.addEventListener('click', function (event) {
                event.preventDefault(); // Prevent the default link behavior

                const deleteUrl = this.getAttribute('data-url'); // Get the delete URL

                // Show SweetAlert confirmation dialog
                Swal.fire({
                    title: 'Are you sure?',
                    text: "You won't be able to revert this!",
                    icon: 'warning',
                    showCancelButton: true,
                    confirmButtonColor: '#3085d6',
                    cancelButtonColor: '#d33',
                    confirmButtonText: 'Yes, delete it!'
                }).then((result) => {
                    if (result.isConfirmed) {
                        // Redirect to the delete URL
                        window.location.href = deleteUrl;
                    }
                });
            });
        });
    });
</script>
```

6)

7)

